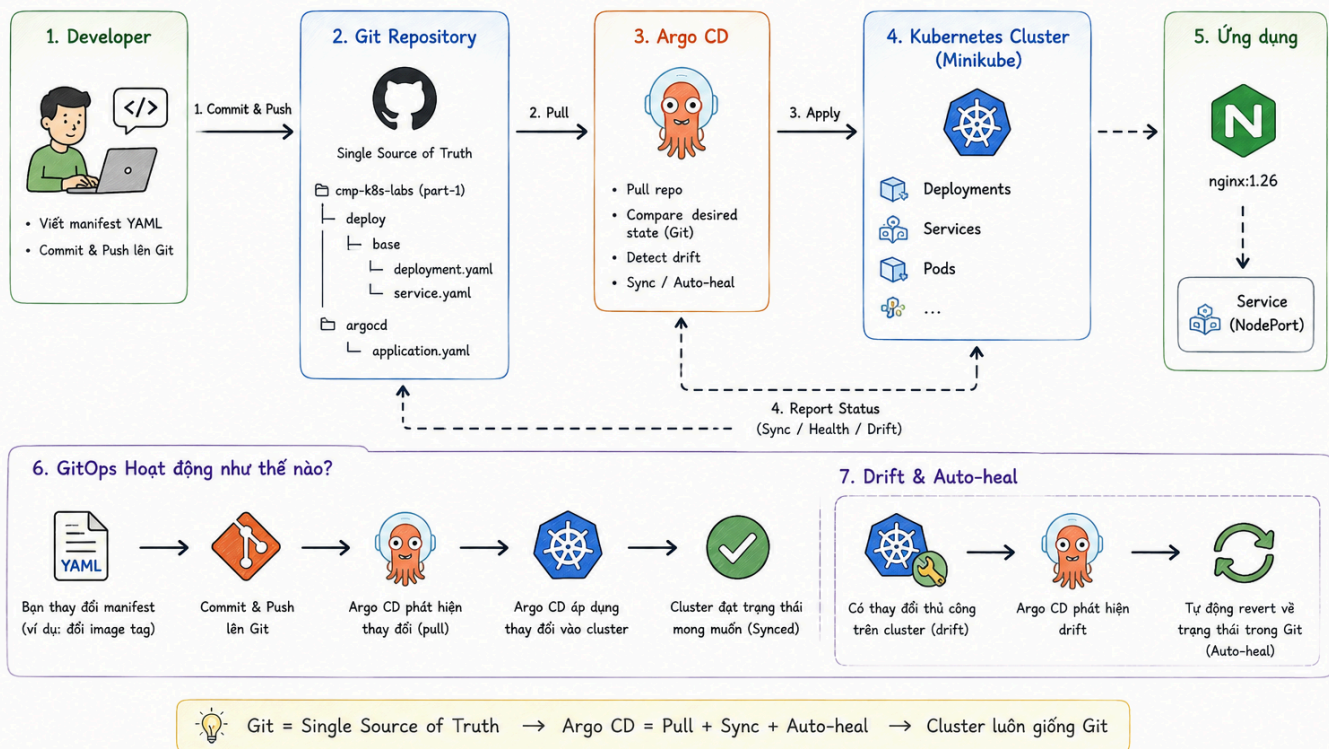


Part 1 — Triển khai ứng dụng bằng Argo CD và quan sát reconciliation

Thời lượng dự kiến: 60–90 phút Kết quả: một ứng dụng nginx được quản lý hoàn toàn từ Git, có thể tự sửa drift và rollback bằng `git revert`.

GitOps với Argo CD trên Minikube (Part 1) – Tổng quan

Git là Single Source of Truth – Argo CD pull và sync vào Kubernetes



1. Mục tiêu học tập

Sau bài này, bạn có thể:

- Giải thích desired state trong Git và live state trong cluster.
- Cài Argo CD, khai báo một `Application` và kiểm tra trạng thái `Synced/Healthy`.
- Deploy, cập nhật và rollback ứng dụng chỉ bằng thay đổi trong Git.
- Phân biệt self-heal và prune qua hai failure drill.
- Dùng `get`, `describe`, `events` và `rollout` để kiểm tra Deployment.

Liên hệ CKA/CKAD

Nội dung	CKAD	CKA	Kỹ năng thực tế
Deployment, Service, rollout, labels/selectors	Trọng tâm	Liên quan	Có
Troubleshooting Pod và Service	Trọng tâm	Trọng tâm	Có
Argo CD và GitOps	Không phải trọng tâm trực tiếp	Không phải trọng tâm trực tiếp	Trọng tâm

Argo CD không phải nội dung chính của kỳ thi. Bài này dùng GitOps làm bối cảnh để luyện các Kubernetes object và lệnh kiểm tra quan trọng.

2. Kiến trúc



Git là nguồn sự thật cho cấu hình được quản lý. Argo CD liên tục so sánh trạng thái trong Git với cluster và reconcile khi hai trạng thái khác nhau.

3. Điều kiện chuẩn bị

Bạn cần:

- Docker hoặc container runtime tương thích với Minikube.
- `git`, `kubectl` và `minikube`.
- Một repository GitHub do bạn sở hữu. Với repository public, Argo CD có thể đọc ngay; với repository private, làm thêm bước cấu hình credential ở phần bên dưới.
- Bash, Git Bash hoặc WSL để chạy nguyên văn các lệnh. Người dùng PowerShell xem ghi chú riêng ở bước lấy password.
- Tối thiểu khoảng 2 CPU và 4 GB RAM còn trống cho Part 1–3.

Kiểm tra công cụ:

```
git --version
kubectl version --client
minikube version
```

Clone repository đã fork và tạo branch làm bài:

```
git clone https://github.com/<your-username>/cmp-k8s-labs.git
cd cmp-k8s-labs
git checkout -b part-1
```

Thay <your-username> một lần trong các file bên dưới. Không copy nguyên placeholder vào Git.

4. Cấu trúc repository

```
cmp-k8s-labs/
├── deploy/
│   ├── base/
│   │   ├── namespace.yaml
│   │   ├── deployment.yaml
│   │   └── service.yaml
│   └── argocd/
│       └── application.yaml
```

5. Thực hành

Bước 1: Tạo cluster

```
minikube start --nodes 1 --driver=docker --cni=calico --memory=4096 --cpus=2
kubectl wait --for=condition=Ready node --all --timeout=180s
kubectl get nodes
```

Tham khảo: <https://blog.cloudmentor.pro/posts/cka-onboarding-chon-playground-tools-va-setup-minikube-de-bat-dau-lab-kubernetes#4-setup-minikube-step-by-step>

Kỳ vọng: node có trạng thái `Ready` .

```

PS C:\Users\USER> minikube start --nodes 1 --driver=docker --cni=calico --memory=4096 --cpus=2
🐳 minikube v1.38.1 on Microsoft Windows 10 Pro 22H2
🌟 Using the docker driver based on user configuration
! Starting v1.39.0, minikube will default to "containerd" container runtime. See #21973 for more info.
🔧 Using Docker Desktop driver with root privileges
👍 Starting "minikube" primary control-plane node in "minikube" cluster
📦 Pulling base image v0.0.50 ...
🔗 Creating docker container (CPUs=2, Memory=4096MB) ...
🔧 Preparing Kubernetes v1.35.1 on Docker 29.2.1 ...
🔗 Configuring Calico (Container Networking Interface) ...
🔍 Verifying Kubernetes components...
  ✨ Using image gcr.io/k8s-minikube/storage-provisioner:v5
  ✨ Enabled addons: storage-provisioner, default-storageclass

! C:\Program Files\Docker\Docker\resources\bin\kubectl.exe is version 1.22.5, which may have incompatibilities with Kubernetes 1.35.1.
  ✨ Want kubectl v1.35.1? Try 'minikube kubectl -- get pods -A'
👍 Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
PS C:\Users\USER> kubectl get nodes
NAME          STATUS    ROLES          AGE   VERSION
minikube      NotReady  control-plane   26s   v1.35.1
PS C:\Users\USER>

```

Bước 2: Cài Argo CD

```

kubectl create namespace argocd
kubectl apply -n argocd --server-side --force-conflicts -f
https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
kubectl wait --for=condition=Available deployment/argocd-server -n argocd --
timeout=300s
kubectl get pods -n argocd

```

--server-side là bắt buộc với manifest Argo CD hiện tại vì một số CRD, đặc biệt ApplicationSet, lớn hơn giới hạn 262144 byte của annotation do client-side apply tạo ra. --force-conflicts cho phép Server-Side Apply nhận quyền quản lý các field cần thiết theo lệnh cài đặt chính thức của Argo CD.

```

networkpolicy.networking.k8s.io/argocd-redis-network-policy serverside-applied
networkpolicy.networking.k8s.io/argocd-repo-server-network-policy serverside-applied
networkpolicy.networking.k8s.io/argocd-server-network-policy serverside-applied
PS C:\Users\USER> kubectl wait --for=condition=Available deployment/argocd-server -n argocd --timeout=300s
deployment.apps/argocd-server condition met
PS C:\Users\USER> kubectl get pods -n argocd
NAME                                READY   STATUS    RESTARTS   AGE
argocd-application-controller-0     1/1     Running   0           3m19s
argocd-applicationset-controller-568dfdf75b-hv6rd  1/1     Running   1 (53s ago)  3m19s
argocd-dex-server-856bcd9ff-4p4hz    1/1     Running   0           3m19s
argocd-notifications-controller-6b4fd8f59-6lbbd  1/1     Running   0           3m19s
argocd-redis-54c57dd6ff-5h2jp       1/1     Running   0           3m19s
argocd-repo-server-fd55df7c-cfr6n    1/1     Running   0           3m19s
argocd-server-6cd5f98457-jv89d      1/1     Running   0           3m19s
PS C:\Users\USER>

```

Kỳ vọng: các Pod chuyển sang Running. Một số thành phần có thể cần thêm thời gian sau khi argocd-server sẵn sàng.

Truy cập UI trong một terminal riêng:

```

kubectl port-forward svc/argocd-server -n argocd 8080:443

```

Mở <https://localhost:8080>. Trình duyệt có thể cảnh báo vì chứng chỉ local.

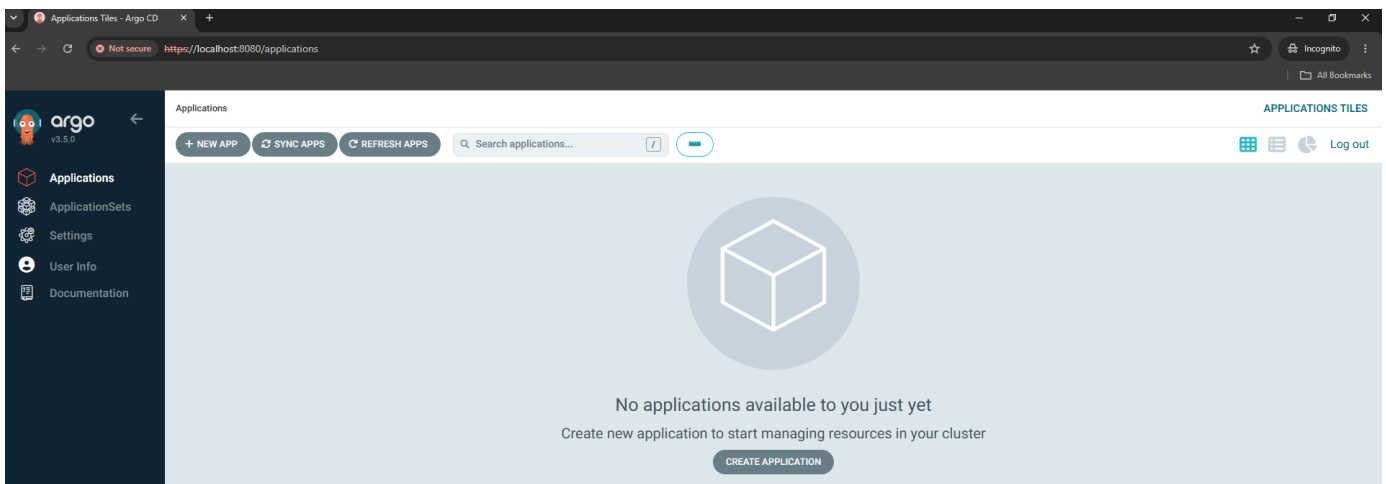
Lấy password trên Bash:

```
kubectl get secret argocd-initial-admin-secret -n argocd -o jsonpath="{.data.password}" | base64 -d  
echo
```

PowerShell:

```
$encodedPassword = kubectl get secret argocd-initial-admin-secret -n argocd -o  
jsonpath="{.data.password}"  
[Text.Encoding]::UTF8.GetString([Convert]::FromBase64String($encodedPassword))
```

Username là `admin` . Không sử dụng password khởi tạo này cho môi trường production.



Kỳ vọng: dashboard chưa có Application nào vì chúng ta mới cài Argo CD; `demo-app` sẽ xuất hiện sau Bước 4.

Bước 3: Tạo Kubernetes manifests

Tạo các thư mục và file theo cấu trúc repository ở trên bằng VS Code, sau đó paste nội dung tương ứng.

`deploy/base/namespace.yaml` :

```
apiVersion: v1  
kind: Namespace  
metadata:  
  name: demo
```

`deploy/base/deployment.yaml` :

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
  namespace: demo
spec:
  replicas: 1
  selector:
    matchLabels:
      app.kubernetes.io/name: nginx
  template:
    metadata:
      labels:
        app.kubernetes.io/name: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.26-alpine
          ports:
            - name: http
              containerPort: 80
          readinessProbe:
            httpGet:
              path: /
              port: http
            initialDelaySeconds: 2
            periodSeconds: 5
          livenessProbe:
            httpGet:
              path: /
              port: http
            initialDelaySeconds: 10
            periodSeconds: 10
          resources:
            requests:
              cpu: 50m
              memory: 32Mi
            limits:
              cpu: 200m
              memory: 128Mi
```

deploy/base/service.yaml :

```
apiVersion: v1
kind: Service
metadata:
  name: nginx
  namespace: demo
spec:
  type: ClusterIP
  selector:
    app.kubernetes.io/name: nginx
  ports:
    - name: http
      port: 80
      targetPort: http
```

Kiểm tra manifest trước khi commit:

```
kubectl apply --dry-run=client -f deploy/base/
```

Bước 4: Tạo Argo CD Application hoàn chỉnh

argocd/application.yaml:

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: demo-app
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/<your-username>/cmp-k8s-labs.git
    targetRevision: part-1
    path: deploy/base
  destination:
    server: https://kubernetes.default.svc
    namespace: demo
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
```

Commit và push desired state:

```
git add deploy argocd
git commit -m "part 1: add demo app and Argo CD application"
git push -u origin part-1
```

Sau khi GitHub đã có các file trên, xác định repository của bạn là public hay private trước khi bootstrap `Application`.

Lưu ý: nếu GitHub repository là private

Flow mặc định của bài đang giả định repository public. Với repository private, Argo CD không thể clone source cho tới khi bạn cấp credential. Nếu dùng HTTPS, `repoURL` trong `Application` vẫn giữ nguyên; credential được cấu hình riêng trong Argo CD và không được ghi trực tiếp vào manifest `Application`. Nếu dùng SSH deploy key, bạn phải đổi `repoURL` sang SSH URL như hướng dẫn bên dưới.

Cách đơn giản nhất cho lab là dùng HTTPS và GitHub fine-grained personal access token:

1. Trên GitHub, tạo fine-grained token chỉ áp dụng cho repository lab.
2. Chỉ cấp quyền repository contents ở mức **Read-only**. Argo CD chỉ cần đọc Git; GitHub Actions dùng credential riêng để push.
3. Trong Argo CD UI, vào **Settings** → **Repositories** → **Connect Repo** → **VIA HTTPS**.
4. Nhập URL đầy đủ, ví dụ `https://github.com/<your-username>/cmp-k8s-labs.git`.
5. Nhập GitHub username và dùng token làm password, sau đó chọn **Connect**.
6. Chỉ tiếp tục khi trạng thái connection là **Successful**.

Nếu đã cài Argo CD CLI và đăng nhập, bạn có thể làm tương đương mà không ghi token trực tiếp vào lịch sử shell:

```
read -s -p "GitHub token: " GITHUB_TOKEN
echo

argocd repo add "https://github.com/<your-username>/cmp-k8s-labs.git" \
  --username "<your-github-username>" \
  --password "$GITHUB_TOKEN"

unset GITHUB_TOKEN
```

Kỳ vọng: CLI báo repository connection thành công. Nếu tổ chức không cho dùng personal access token, có hai lựa chọn tốt hơn.

Lựa chọn SSH deploy key

Deploy key là một SSH key chỉ cấp quyền cho **một repository** và mặc định có thể để read-only. Cách làm ngắn gọn:

1. Tạo một key riêng, không dùng lại SSH key cá nhân:

```
ssh-keygen -t ed25519 \  
-C "argocd-cmp-k8s-labs" \  
-f ./argocd-cmp-k8s-labs \  
-N ""
```

Lệnh tạo private key `argocd-cmp-k8s-labs` và public key `argocd-cmp-k8s-labs.pub`. Key không có passphrase vì Argo CD phải sử dụng không tương tác; vì vậy cần bảo vệ private key cẩn thận.

- Trong GitHub repository, vào **Settings** → **Deploy keys** → **Add deploy key**. Đặt tên để nhận biết và paste nội dung file `.pub`:

```
cat ./argocd-cmp-k8s-labs.pub
```

- Không chọn “Allow write access”**. Argo CD chỉ cần clone/read repository.
- Kết nối repository với Argo CD bằng private key:

```
argocd repo add "git@github.com:<your-username>/cmp-k8s-labs.git" \  
--ssh-private-key-path ./argocd-cmp-k8s-labs
```

- Đổi `repoURL` trong `argocd/application.yaml` cho khớp chính xác SSH URL:

```
repoURL: git@github.com:<your-username>/cmp-k8s-labs.git
```

- Khi Argo CD báo connection thành công, xóa private key khỏi máy lab nếu không còn cần giữ ở đó, hoặc chuyển nó vào secret manager an toàn. Tuyệt đối không chạy `git add` với hai file key.

Tham khảo: [GitHub — Managing deploy keys](#) và [Argo CD — Private repositories/SSH](#).

Lựa chọn GitHub App

GitHub App phù hợp khi quản lý nhiều repository; cấp tối thiểu quyền **Contents: Read-only** và chỉ install App vào các repository cần thiết. Xem [hướng dẫn GitHub App credential của Argo CD](#).

Không commit token, password hoặc SSH private key vào repository — kể cả dưới dạng base64 trong Kubernetes Secret. Với môi trường production, nên dùng GitHub App hoặc secret manager/External Secrets/Sealed Secrets và có quy trình rotate/revoke credential. Không dùng `--insecure-skip-server-verification` để xử lý lỗi credential hay TLS trong production.

Tài liệu tham khảo: [Argo CD — Private Repositories](#) và [Argo CD CLI — `argocd repo add`](#).

Sau khi repository public đã sẵn sàng hoặc private repository đã kết nối thành công, bootstrap `Application`:

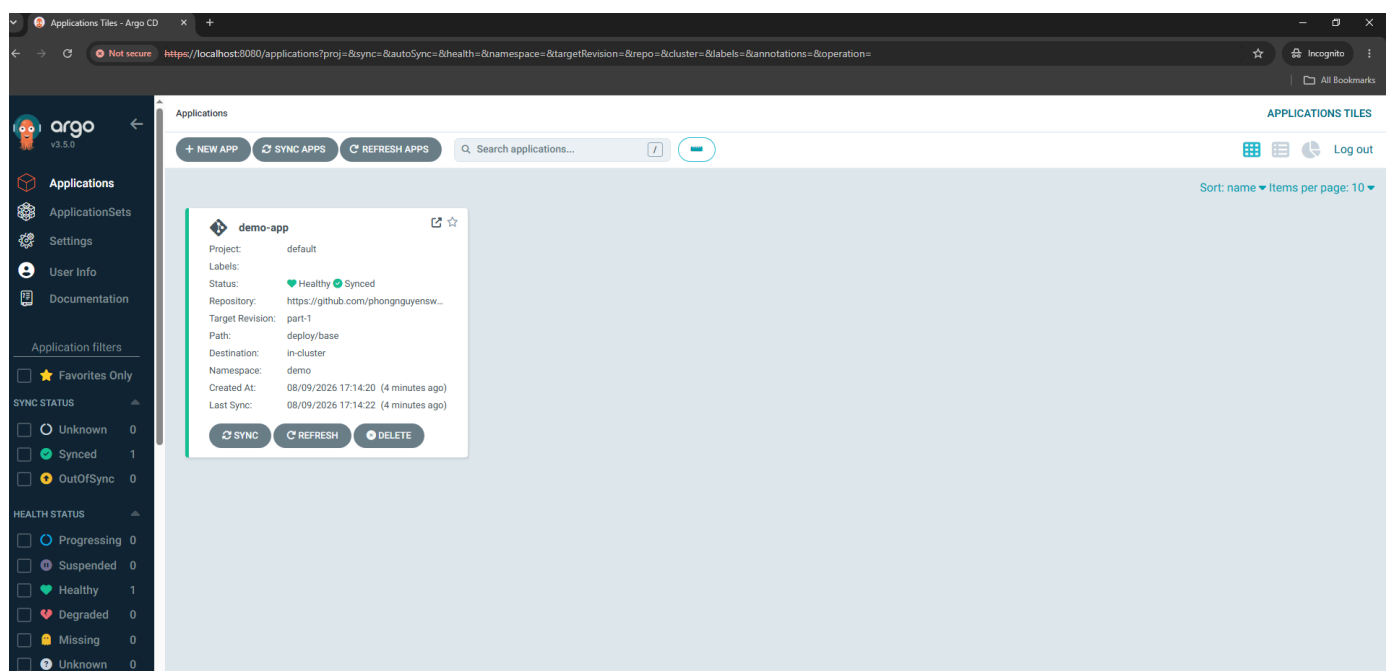
```
kubectl apply -f argocd/application.yaml
kubectl get applications -n argocd
kubectl get application demo-app -n argocd -w
```

```
PS F:\cmp-k8s-labs> kubectl apply -f argocd/application.yaml
application.argoproj.io/demo-app created
PS F:\cmp-k8s-labs> kubectl get applications -n argocd
NAME          SYNC STATUS  HEALTH STATUS
demo-app
PS F:\cmp-k8s-labs> kubectl get application demo-app -n argocd -w
NAME          SYNC STATUS  HEALTH STATUS
demo-app
demo-app
demo-app      OutOfSync    Missing
demo-app      OutOfSync    Missing
demo-app      OutOfSync    Missing
demo-app      OutOfSync    Missing
demo-app      OutOfSync    Missing
demo-app      Synced       Progressing
demo-app      Synced       Progressing
demo-app      Synced       Healthy
```

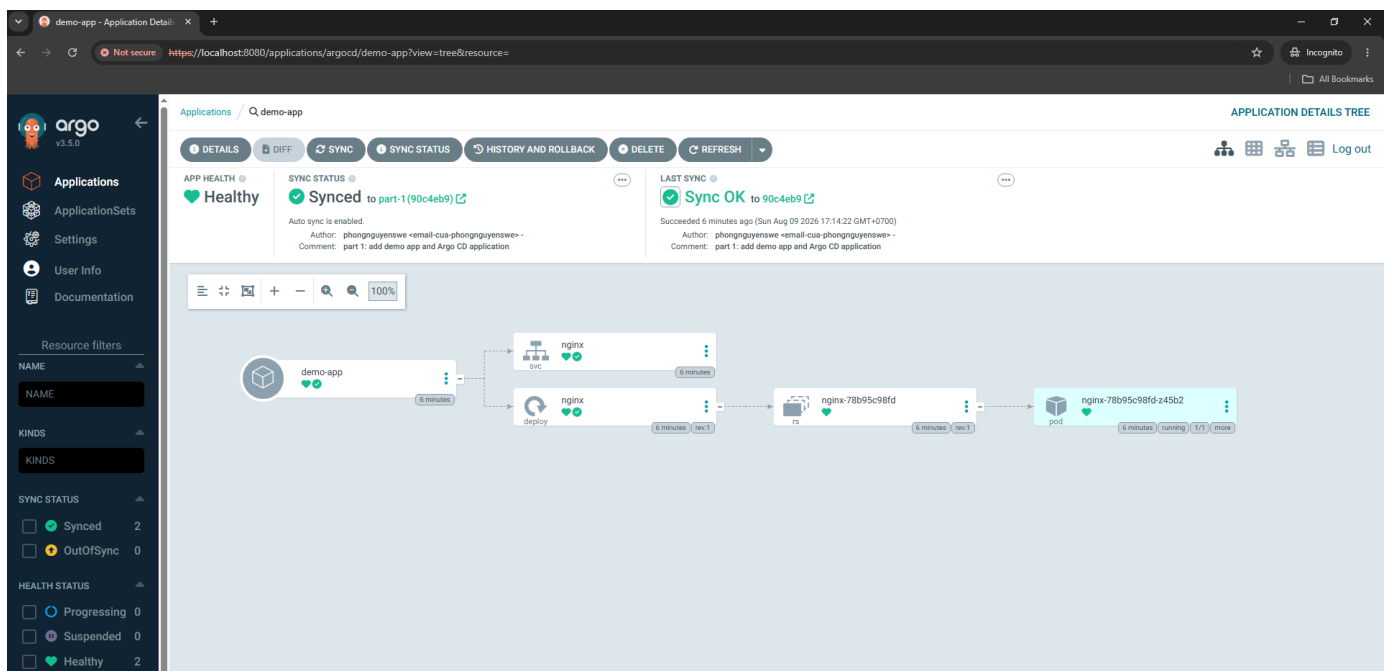
Kỳ vọng: Application lần lượt chuyển từ `OutOfSync/Missing` sang `Synced/Progressing`, cuối cùng đạt `Synced/Healthy` khi toàn bộ workload sẵn sàng.

Dừng theo dõi khi thấy `Synced` và `Healthy`.

Mở lại Argo CD dashboard. Application `demo-app` lúc này hiển thị hai trạng thái `Healthy` và `Synced`:



Chọn `demo-app` để xem resource tree. Argo CD thể hiện quan hệ từ Application tới Service, Deployment, ReplicaSet và Pod:



Bước 5: Kiểm tra ứng dụng

```
kubectl rollout status deployment/nginx -n demo --timeout=120s
kubectl get pods,service,endpointslice -n demo
kubectl get deployment nginx -n demo -o
jsonpath='{.spec.template.spec.containers[0].image}'; echo
```

Truy cập app trong terminal riêng:

```
kubectl port-forward service/nginx -n demo 8081:80
```

Sau đó:

```
curl http://localhost:8081
```

Kỳ vọng: nhận trang mặc định của nginx.



Welcome to nginx!

If you see this page, the nginx web server is successfully installed and working. Further configuration is required.

For online documentation and support please refer to nginx.org.
Commercial support is available at nginx.com.

Thank you for using nginx.

Bước 6: Cập nhật qua Git

Mục tiêu của bước này là nâng **phiên bản của cùng ứng dụng nginx**, không phải đổi sang web server khác như `httpd`. Ví dụ, sửa `deploy/base/deployment.yaml`:

```
image: nginx:1.27-alpine
```

Trước khi push, cluster vẫn chạy version cũ:

```
kubectl get deployment nginx -n demo -o  
jsonpath='{.spec.template.spec.containers[0].image}'; echo
```

Commit desired state mới lên branch `part-1`:

```
git add deploy/base/deployment.yaml  
git commit -m "part 1: update nginx to 1.27-alpine"  
git push
```

Vì `Application` đã bật automated sync, Argo CD sẽ:

1. Phát hiện branch `part-1` có commit mới.
2. So sánh Git với live state và thấy Deployment khác image.
3. Tự động apply desired state mới vào cluster.
4. Kubernetes tạo ReplicaSet và Pod mới theo rolling update, sau đó xóa Pod cũ.

Theo dõi Argo CD cho tới khi Application trở lại `Synced/Healthy`:

```
kubectl get application demo-app -n argocd -w
```

Nhấn `Ctrl+C` khi đã thấy `Synced/Healthy`, rồi kiểm tra rollout và image thực tế:

```
kubectl rollout status deployment/nginx -n demo --timeout=180s
kubectl get replicaset,pods -n demo
kubectl get deployment nginx -n demo -o
jsonpath='{.spec.template.spec.containers[0].image}'; echo
```

Kỳ vọng: image đang chạy là `nginx:1.27-alpine`, Pod mới ở trạng thái `Running`, Application là `Synced/Healthy`.

```
PS F:\cmp-k8s-labs> kubectl rollout status deployment/nginx -n demo --timeout=180s
deployment "nginx" successfully rolled out
PS F:\cmp-k8s-labs> kubectl get replicaset,pods -n demo
NAME                                DESIRED    CURRENT    READY    AGE
replicaset.apps/nginx-67967bf85      1          1          1        75s
replicaset.apps/nginx-78b95c98fd     0          0          0        19m

NAME                                READY    STATUS    RESTARTS    AGE
pod/nginx-67967bf85-kdt6f            1/1      Running   0           75s
PS F:\cmp-k8s-labs> kubectl get deployment nginx -n demo -o jsonpath='{.spec.template.spec.containers[0].image}';
nginx:1.27-alpine
PS F:\cmp-k8s-labs>
```

Trong kết quả trên, ReplicaSet mới có `DESIRED/CURRENT/READY = 1`, ReplicaSet cũ đã scale về `0`, Pod mới đang `Running` và Deployment sử dụng image `nginx:1.27-alpine`.

Bạn vẫn có thể thử đổi sang `httpd`, nhưng khi đó đang thay đổi ứng dụng chứ không chỉ nâng version. Cần kiểm tra lại container name, port, probe và nội dung trả về. Để minh họa GitOps rõ ràng và ít phát sinh lỗi ngoài mục tiêu, bài này chỉ đổi version `nginx`.

Bước 7: Failure drill — self-heal

Tạo drift trực tiếp trong cluster:

```
kubectl scale deployment nginx -n demo --replicas=3
kubectl get deployment nginx -n demo -w
```

Kỳ vọng: Argo CD phát hiện drift và đưa `replicas` về `1`.

```
PS F:\cmp-k8s-labs> kubectl scale deployment nginx -n demo --replicas=3
deployment.apps/nginx scaled
PS F:\cmp-k8s-labs> kubectl get deployment nginx -n demo -w
NAME    READY    UP-TO-DATE    AVAILABLE    AGE
nginx   1/3      3             1            23m
nginx   1/1      3             1            23m
nginx   1/1      3             1            23m
nginx   1/1      3             1            23m
nginx   1/1      1             1            23m
```

Trong ảnh, `kubectl scale` tạm thời đưa `DESIRED` lên `3`; sau khi Argo CD reconcile, desired state được khôi phục về `1` đúng với manifest trong Git.

Nếu chưa thấy thay đổi, kiểm tra:

```
kubectl describe application demo-app -n argocd
kubectl get events -n argocd --sort-by=.lastTimestamp
```

Bước 8: Kiểm tra prune — xóa resource không còn trong Git

Ở Bước 7, Deployment **vẫn còn trong Git** nhưng bị sửa trực tiếp trên cluster, nên `selfHeal` đưa nó về đúng cấu hình.

Bước này kiểm tra tình huống khác: một resource từng được Argo CD quản lý **không còn tồn tại trong Git**. Với `prune: true`, Argo CD hiểu resource đó không còn thuộc desired state và xóa nó khỏi cluster.

```
Thêm file vào Git → Argo CD tạo ConfigMap trong cluster
Xóa file khỏi Git → Argo CD prune ConfigMap khỏi cluster
```

8.1. Thêm một resource tạm thời

Tạo file `deploy/base/banner.yaml`:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: temporary-banner
  namespace: demo
data:
  message: temporary
```

```
git add deploy/base/banner.yaml
git commit -m "part 1: add temporary config"
git push
```

Theo dõi Application cho tới khi trở lại `Synced/Healthy`:

```
kubectl get application demo-app -n argocd -w
```

Nhấn `Ctrl+C`, sau đó xác nhận Argo CD đã tạo ConfigMap:

```
kubectl get configmap temporary-banner -n demo
```

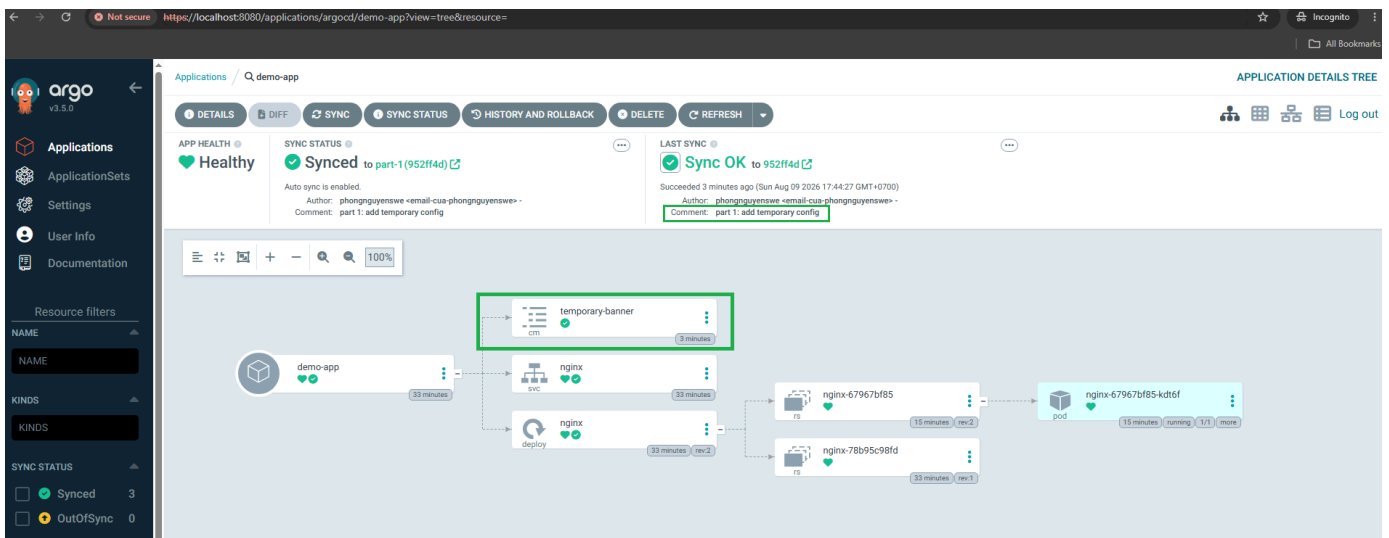
Kỳ vọng: ConfigMap `temporary-banner` tồn tại trong cluster.

```
PS F:\cmp-k8s-labs> kubectl get application demo-app -n argocd -w
NAME          SYNC STATUS  HEALTH STATUS
demo-app      Synced       Healthy

PS F:\cmp-k8s-labs> kubectl get configmap temporary-banner -n demo
NAME          DATA  AGE
temporary-banner  1      21s
```

Ảnh cho thấy Application vẫn **Synced/Healthy** và ConfigMap **temporary-banner** đã xuất hiện trong namespace **demo** với một key dữ liệu.

Trên Argo CD UI, ConfigMap cũng xuất hiện trong resource tree và được đánh dấu đã đồng bộ:



8.2. Xóa resource khỏi desired state

Xóa file bằng Git, commit và push:

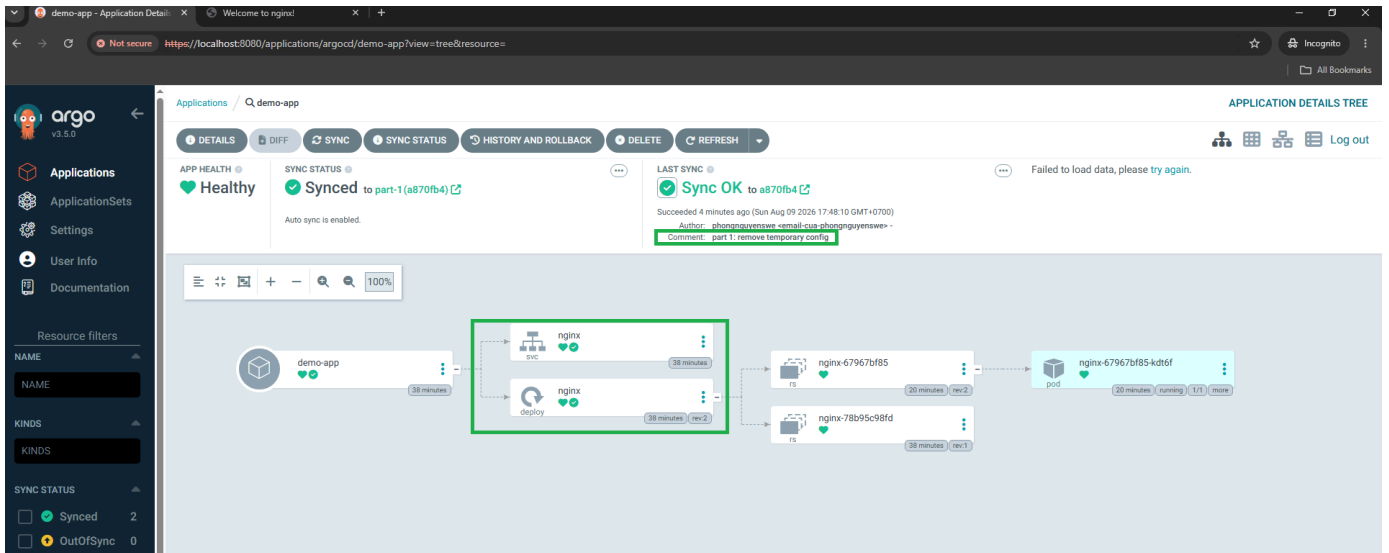
```
git rm deploy/base/banner.yaml
git commit -m "part 1: remove temporary config"
git push
```

Chờ ConfigMap bị prune:

```
kubectl wait --for=delete configmap/temporary-banner -n demo --timeout=180s
kubectl get configmap temporary-banner -n demo
```

Kỳ vọng:

- Lệnh `kubectl wait` báo resource đã bị xóa.
- Lệnh `kubectl get` cuối cùng trả về **NotFound** — đây là kết quả đúng của bài test.
- Application trở lại **Synced/Healthy**.



Sau khi file `banner.yaml` bị xóa khỏi Git, ConfigMap `temporary-banner` đã được Argo CD xóa khỏi cluster và không còn xuất hiện trong resource tree. Application vẫn `Synced/Healthy`; tree chỉ còn các resource thuộc desired state hiện tại như Service, Deployment, ReplicaSet và Pod.

`prune` không có nghĩa Argo CD xóa mọi resource lạ trong namespace. Nó chỉ prune resource thuộc Application mà trước đó Argo CD theo dõi, nhưng nay đã bị loại khỏi desired state trong Git.

Tóm lại:

Tình huống	Tính năng xử lý
Resource vẫn có trong Git nhưng live state bị sửa	<code>selfHeal</code>
Resource từng được quản lý nhưng đã bị xóa khỏi Git	<code>prune</code>

Bước 9: Rollback bằng Git

Giả sử sau khi nâng lên `nginx:1.27-alpine` ở Bước 6, phiên bản mới gặp lỗi và bạn muốn quay lại `nginx:1.26-alpine`.

Trong GitOps, không nên chạy:

```
kubectl rollout undo deployment/nginx -n demo
```

Lệnh trên chỉ sửa live state trong cluster, trong khi Git vẫn khai báo `nginx:1.27-alpine`. Argo CD sẽ phát hiện drift và có thể self-heal cluster trở lại version `1.27`.

Cách đúng là rollback **desired state trong Git**.

9.1. Tìm commit đã nâng image

Chỉ xem lịch sử thay đổi của Deployment:

```
git log --oneline -- deploy/base/deployment.yaml
```

Tìm commit có message:

```
part 1: update nginx to 1.27-alpine
```

Copy commit SHA ở đầu dòng, ví dụ `abc1234`.

9.2. Revert commit cập nhật image

Thay `<image-update-commit-sha>` bằng SHA vừa tìm được:

```
git revert --no-edit <image-update-commit-sha>
git push
```

`git revert` không xóa hay sửa lịch sử cũ. Git tạo một commit mới có nội dung đảo ngược thay đổi của commit nâng image, vì vậy `deployment.yaml` trở lại `nginx:1.26-alpine`.

Kiểm tra Git trước khi chờ Argo CD:

```
git grep 'image:' -- deploy/base/deployment.yaml
git log --oneline --max-count=3
```

9.3. Chờ Argo CD rollback cluster

Theo dõi Application cho tới khi trở lại `Synced/Healthy`:

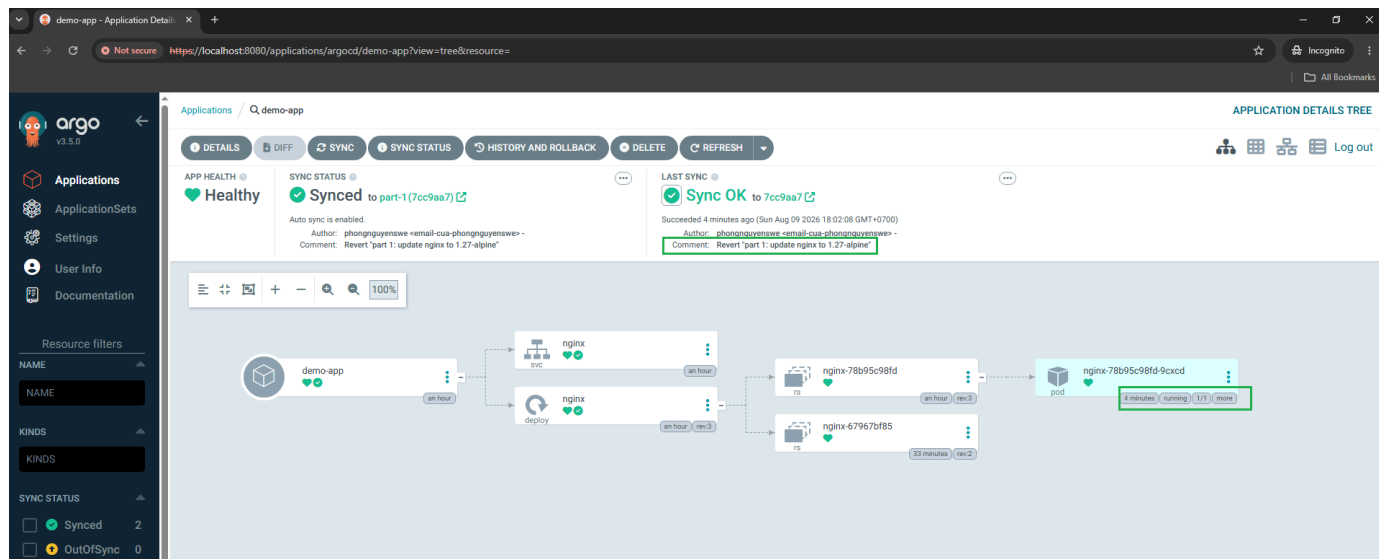
```
kubectl get application demo-app -n argocd -w
```

Nhấn `Ctrl+C`, sau đó kiểm tra rollout và image:

```
kubectl rollout status deployment/nginx -n demo --timeout=180s
kubectl get deployment nginx -n demo -o
jsonpath='{.spec.template.spec.containers[0].image}'; echo
kubectl get replicaset,pods -n demo
```

Kỳ vọng:

- Git chứa một revert commit mới.
- Deployment chạy lại `nginx:1.26-alpine`.
- Kubernetes tạo rollout mới để thay Pod `1.27` bằng Pod `1.26`.
- Argo CD trở lại `Synced/Healthy`.



Trong ảnh, Last Sync trở tới commit Revert "part 1: update nginx to 1.27-alpine", Application đã trở lại Synced/Healthy và Pod mới được tạo từ desired state sau rollback. Điều này chứng minh rollback được thực hiện qua Git và vẫn giữ đầy đủ audit trail.

```
Git revert 1.27 → desired state trở về 1.26
```



```
Argo CD sync
```



```
Cluster rollout từ 1.27 về 1.26
```

Đây là rollback theo GitOps: Git vẫn là source of truth và toàn bộ thay đổi có audit trail.

6. Troubleshooting nhanh

Triệu chứng	Lệnh kiểm tra	Nguyên nhân thường gặp
Application Unknown	<code>kubectl describe application demo-app -n argocd</code>	Sai URL, branch, path hoặc repo private
Pod ImagePullBackOff	<code>kubectl describe pod -n demo <pod></code>	Tag image không tồn tại hoặc registry cần credential
Service không trả response	<code>kubectl get endpointslice -n demo</code>	Selector không khớp label hoặc Pod chưa Ready
App không self-heal	<code>kubectl get application demo-app -n argocd -o yaml</code>	Thiếu automated sync/selfHeal
<code>applicationsets.argoproj.io ... annotations: Too long</code>	Chạy lại lệnh cài với <code>--server-side --force-conflicts</code>	Client-side apply vượt giới hạn annotation của CRD

Ưu tiên đọc `Events` và condition trước khi xóa/recreate tài nguyên.

7. Definition of done

Nếu repository lab đã chứa bộ verifier, chạy từ repository root:

```
bash scripts/verify-part-1.sh
```

Trên Windows PowerShell, gọi đúng Git Bash thay vì `C:\Windows\System32\bash.exe` của WSL:

```
& "C:\Program Files\Git\bin\bash.exe" scripts/verify-part-1.sh
```

```
PS F:\k8s-mentor\GitOps\series_blog> & "C:\Program Files\Git\bin\bash.exe" scripts/verify-part-1.sh
INFO  Kubernetes context: minikube
PASS  Namespace demo exists
PASS  Argo CD sync status = Synced
PASS  Argo CD health status = Healthy
PASS  Automated prune = true
PASS  Automated self-heal = true
PASS  Deployment nginx rollout is complete
PASS  Available replicas = 1
PASS  Deployment image: nginx:1.26-alpine
PASS  Service nginx has 1 endpoint(s)
INFO  HTTP check skipped; set VERIFY_URL after starting a port-forward

Part 1 verification passed.
```

Kỳ vọng: toàn bộ Kubernetes và Argo CD checks hiển thị `PASS`, dòng cuối là `Part 1 verification passed.`. HTTP check có thể hiển thị `skipped` nếu bạn chưa truyền `VERIFY_URL`; điều này không làm verifier thất bại.

- ☐ Argo CD Application hiển thị `Synced` và `Healthy`.
- ☐ `curl` trả về trang nginx.
- ☐ Image thay đổi qua commit Git và rollout hoàn tất.
- ☐ Scale trực tiếp bị self-heal về desired state.
- ☐ Tài nguyên bị bỏ khỏi Git được prune khỏi cluster.
- ☐ Rollback thực hiện bằng `git revert`.

8. Cleanup

Nếu tiếp tục Part 2, giữ cluster và Argo CD. Nếu dừng series:

```
kubectl delete application demo-app -n argocd
minikube delete
```

9. Challenge

Không xem lại manifest mẫu, hãy sửa selector của Service làm mất EndpointSlice, dùng lệnh Kubernetes để tìm nguyên nhân, rồi sửa bằng Git. Mục tiêu thời gian: 10 phút.

Sau khi hoàn thành hoặc hết thời gian, đối chiếu với [lời giải Challenge Part 1](#).

10. Bài tiếp theo

Ở Part 1, bạn vẫn phải tự sửa image nginx trong file YAML rồi push lên Git.

Trong Part 2, chúng ta sẽ tự động hóa công việc này:

```
Bạn sửa code và push lên GitHub
    ↓
GitHub Actions tự build Docker image và push lên Docker Hub
    ↓
Pipeline cập nhật image tag trong Kubernetes manifest
    ↓
Argo CD phát hiện thay đổi và deploy phiên bản mới
```

Mỗi Docker image sẽ dùng một tag riêng dựa trên Git commit, thay vì ghi đè lên tag `latest`. Nhờ vậy, bạn có thể biết chính xác source code nào đang chạy trong cluster và rollback khi có lỗi.

Part 2 cũng sẽ chỉ ra cách cấu hình workflow để commit cập nhật manifest không tự kích hoạt thêm một pipeline mới liên tục.

Trong thời gian tới, team mình sẽ triển khai khóa học CKA, CKAD với nhiều lab thực chiến tương tự — bạn có thể tham khảo thêm tại: <https://cloudmentor.pro/courses/cka>

Chúc bạn học tập hiệu quả và sớm chinh phục được chứng chỉ CKA! 🍌